SMU / CS610 FOUNDATIONS OF APPLIED MACHINE LEARNING

NYC Citi Bike Demand Forecasting

Predicting hourly bike rental demand to support smarter fleet planning

Project proposal for Group 1
Alessandro Pagan
Alice Agazzi
Lee Jing Hua Amelia
Robin Thomas
Suhasini Pillai Do Sivashankar

BUSINESS PROBLEM

Why Forecast Bike Demand?

Citi Bike loses revenue when customers are unable to get bikes at some stations due to a shortage, while bikes sit idle at other stations. Using machine learning to predict demand at each station helps Citi Bike move bikes more effectively to where they'll be needed, reducing customer frustration and lost trips.

Without Machine Learning

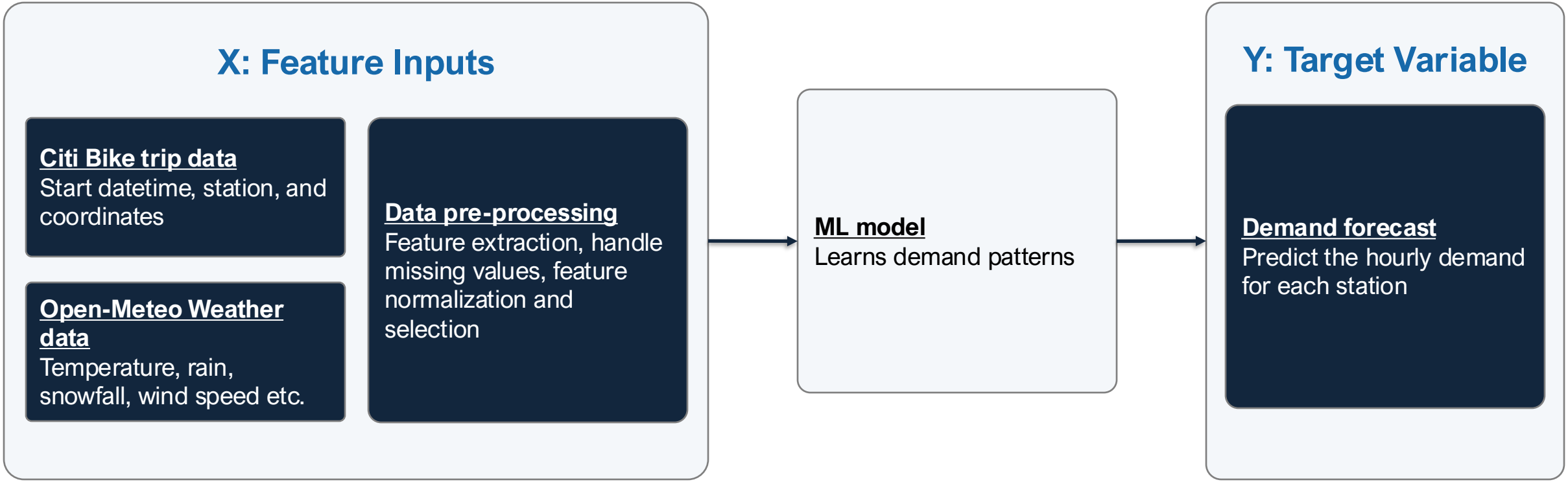
- Bikes are moved based on fixed schedules, simple historical averages and/or staff judgement.
- Fewer factors are considered, making it harder to adapt to changing conditions.

With Machine Learning

- A model predicts demand at each station using historical usage and more current features such as weather..
- More factors are considered together, allowing faster and better responses to changing conditions.

METHODOLOGY

A Time Series Forecasting Approach



This is a **regression** problem:
Historical trip data and current weather data are used to predict the number of bike rentals at each station in the next hour.

Data Preprocessing Approach

Citi Bike Trip Data

[Source: Kaggle]

Only the rental start point data is used:
- Datetime (discarded later)
- Station name (one-hot encoded)
- Station coordinates (discarded later)

Target variable from summation of rows:
- Number of rentals for each station at each hour (Y)

Hourly Weather Data

[Source: Open-Meteo Historical API]

T-1 data extracted using date and coordinates:
- Temperature at 2m
- Rain
- Snow depth
- Snowfall
- Relative humidity at 2m
- Precipitation
- Wind speed at 10m
- Wind gusts at 10m
- Cloud cover

Note: we assume that the prediction is made 1 hour ahead.

Engineered Features

[Created from trip and weather data]

Extracted from datetime:
- Year
- Month (one-hot encoded)
- Week of year (one-hot encoded)
- Day of week (one-hot encoded)
- Hour of day (one-hot encoded)
- Is public holiday

Extracted from latest rental data:
- number of rentals in the last 3 hours (Y-1, Y-2 and Y-3)

Train-test-split:

TRAIN: Jan - Dec 2020 447,984 rows	Validate: Jan - Feb 2021 72,216 rows	Test: Mar - Apr 2021 74,664 rows

EVALUATION

Metrics Used

Time-ordered split: models learn from Jan-Dec 2020 data , are tuned on Jan–Feb 2021, and are scored once on Mar–Apr 2021, so no future data leaks into training. Every model is compared with simple baselines.

Mean Absolute Error (MAE)

Primary metric

What: average absolute error, in trips per station-hour.

Why: easy for operations to interpret and not dominated by rare spikes.

Root Mean Squared Error (RMSE)

Large-miss check

What: squares errors before averaging, so big misses weigh more.

Why: large under-forecasts at rush hour are what leave stations empty.

Poisson Deviance

Count-data fit

What: likelihood-based error for non-negative counts.

Why: hourly rentals per station are small, skewed counts with many zeros. Used for tuning and model selection.

Naive Baseline

Benchmark for all metrics

What: seasonal-naive forecast (same hour last week) scored on the same test set. New stations use last hour's rentals.

Why: a model must beat this non-ML benchmark to be useful.

How we report: overall, split by peak vs. off-peak hours, and busy vs. quiet stations. We avoided MAPE because many station-hours have zero rentals, which makes percentage error undefined.

ML APPROACH

Machine Learning Techniques

Why these models: the target is a low-count hourly series, and features are a mix of one-hot calendar and station columns, lags and weather. Tree-based ensembles are our main candidates, with linear and neural models for contrast.

Ridge / Poisson Regression

Linear baseline

Simple and interpretable starting point. The Poisson link respects non-negative counts.

Random Forest

Bagged trees

Captures non-linear interactions such as hour × weather with little tuning.

Lightgbm

Gradient boosting

Fast on many one-hot and lag features, with a Poisson objective for counts. Our main candidate.

Neural Network (MLP)

Station embeddings

Learns a compact vector per station instead of hundreds of one-hot columns, sharing patterns across stations.

Ensemble

Stacking / averaging

Blends the models so their different errors partly cancel out.

Reference baselines: seasonal naive (same hour last week, falling back to last hour's rentals) and historical hourly average. These are for comparison and are not counted among the ML techniques.

Our Next Steps / Future Considerations

- Move on to :
  - Feature selection to remove highly correlated features
  - Iteratively train and evaluate the models
  - Select the final model

- We are currently intending to predict the demand on an hourly basis, but may switch to 2-hourly / half-day basis.